

Technical Design Document

Platform, delivery pipeline and security design

Document ID	OBE-TDD-001
Version	1.1
Date	2026-10-10
Status	Approved for the dev environment
Owner	khaipd18 (DevOps / Cloud)
Audience	Cloud and DevOps engineers, security reviewers, maintainers
Classification	Internal
Repository	https://github.com/khaipd18/online-boutique-gitops-pipeline

Document control

Revision history

Version	Date	Change	Author
1.0	2026-10-09	First issue: covers infrastructure, Kubernetes platform, CI/CD, security and the decisions behind them.	khaipd18
1.1	2026-10-10	First run on the new account: state bucket renamed, control plane log group managed by Terraform (74 resources), apply and destroy verified.	khaipd18

Related documents

- OBE-RUN-001 Operations Runbook ([docs/manuals/operations-runbook.en.pdf](#))
- README.md: project summary, verification results and local test guide
- docs/infrastructure/: editable HLD/LLD diagrams (.drawio) and their YAML specs

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Current state	3
1.4	Conventions	3
2	System overview	4
2.1	Application	4
2.2	What this repository adds	4
2.3	High-level architecture	5
3	Requirements and constraints	6
3.1	Non-functional requirements	6
3.2	Constraints	6
4	AWS infrastructure design	7
4.1	Network	7
4.2	Security groups	8
4.3	Compute: Amazon EKS	8
4.4	Container registry: Amazon ECR	9
4.5	Identity and access	9
4.6	Terraform state	10
4.7	Infrastructure as Code structure	10
5	Kubernetes platform design	11
5.1	Namespaces and Pod Security	11
5.2	Universal Helm chart	11
5.3	Service-to-service traffic	12
5.4	Resilience	12
5.5	Observability	12
6	Delivery pipeline design	13
6.1	Continuous integration	13
6.2	Continuous delivery	14
6.3	Infrastructure pipeline	14
6.4	Repository controls	14
7	Security design	15
7.1	Controls by risk	15
7.2	Accepted exceptions	15
7.3	Supply chain	16
8	Design decisions	17
9	Verification	18
10	Risks, limitations and roadmap	19
10.1	Known risks	19
10.2	Roadmap	19
	Appendix A. Versions	20
	Appendix B. Glossary	21

1 Introduction

1.1 Purpose

This document describes how the Online Boutique platform is built on AWS: the infrastructure, the Kubernetes platform, the delivery pipeline and the security controls, and why each design choice was made. It is the reference for anyone who changes the platform. Day-to-day procedures (deploying, releasing, troubleshooting) are in the Operations Runbook (OBE-RUN-001).

1.2 Scope

In scope:

- AWS infrastructure in account-agnostic form, region `ap-southeast-1`: VPC, EKS, ECR, VPC endpoints, IAM roles for GitHub Actions, Terraform state backend.
- Kubernetes platform in namespace `dev-eks`: Helm chart, Argo CD configuration, Pod Security, NetworkPolicy, autoscaling, monitoring stack.
- CI/CD: GitHub Actions workflows, quality and security gates, GitOps write-back, Terraform pipeline, repository controls.

Out of scope:

- Application code under `src/` and `proto/`. It is Google's Online Boutique (Apache License 2.0) and is kept unchanged; findings that can only be fixed in that code are reported, not fixed.
- Staging and production environments. Only one environment (`dev`) exists; Section 10.2 lists what a production environment would add.

1.3 Current state

The platform ran on Amazon EKS in May 2026 (Terraform built the infrastructure, CI pushed images to ECR over OIDC, Argo CD synced the services). The AWS environment was then torn down to avoid cost and the original account is no longer used. The security and resilience layers added afterwards were verified on a local kind cluster and with a read-only `terraform plan`. On 2026-10-10 the current code was applied to a new account and destroyed again, both without errors. Section 9 gives the details.

1.4 Conventions

- `monospace` marks names that exist in the repository or in AWS: files, resources, commands.
- Release names in the cluster are `<service>-dev` (for example `frontend-dev`); tables list the service part only.
- Links in brackets point to the AWS documentation that a statement is based on.

2 System overview

2.1 Application

Online Boutique is an e-commerce demo of 10 microservices written in 5 languages that call each other over gRPC. A user browses products, adds them to a cart stored in Redis, and checks out; checkout calls payment, shipping, email, currency and cart.

Table 1: Services deployed to dev-eks (from `gitops/dev-eks/values-*.yaml`)

Service	Language	Port	Accepts calls from	CPU req/limit	Memory req/limit
frontend	Go	80 → 8080	any (LoadBalancer)	100m / 200m	64Mi / 256Mi
checkoutservice	Go	5050	frontend	100m / 200m	64Mi / 128Mi
productcatalogservice	Go	3550	frontend, checkout, recommendation	100m / 200m	64Mi / 128Mi
shippingservice	Go	50051	frontend, checkout	100m / 200m	64Mi / 128Mi
cartservice	C#	7070	frontend, checkout	200m / 300m	64Mi / 128Mi
adservice	Java	9555	frontend	200m / 300m	180Mi / 300Mi
currencyservice	Node.js	7000	frontend, checkout	100m / 200m	64Mi / 128Mi
paymentservice	Node.js	50051	checkout	100m / 200m	64Mi / 128Mi
emailservice	Python	8080	checkout	100m / 200m	64Mi / 128Mi
recommendationservice	Python	8080	frontend	100m / 200m	250Mi / 500Mi
redis-cart	Redis	6379	cartservice	100m / 200m	64Mi / 128Mi

2.2 What this repository adds

The repository adds everything around the application that is needed to run it on AWS:

Layer	Content
Infrastructure as Code	Terraform root module and 5 modules (<code>vpc</code> , <code>eks</code> , <code>ecr</code> , <code>vpc-endpoints</code> , <code>github-oidc-role</code>).
Packaging	One universal Helm chart (<code>helm-charts/</code>) shared by all 12 releases.
Continuous delivery	Argo CD ApplicationSet that generates one Application per service from <code>gitops/dev-eks/</code> .
Continuous integration	5 per-language workflows, a Terraform workflow and a security scan workflow.
Security	OIDC instead of access keys, Checkov, Trivy, Pod Security Admission, NetworkPolicy.
Documentation	README (English and Vietnamese), HLD/LLD diagrams, this document and the runbook.

2.3 High-level architecture

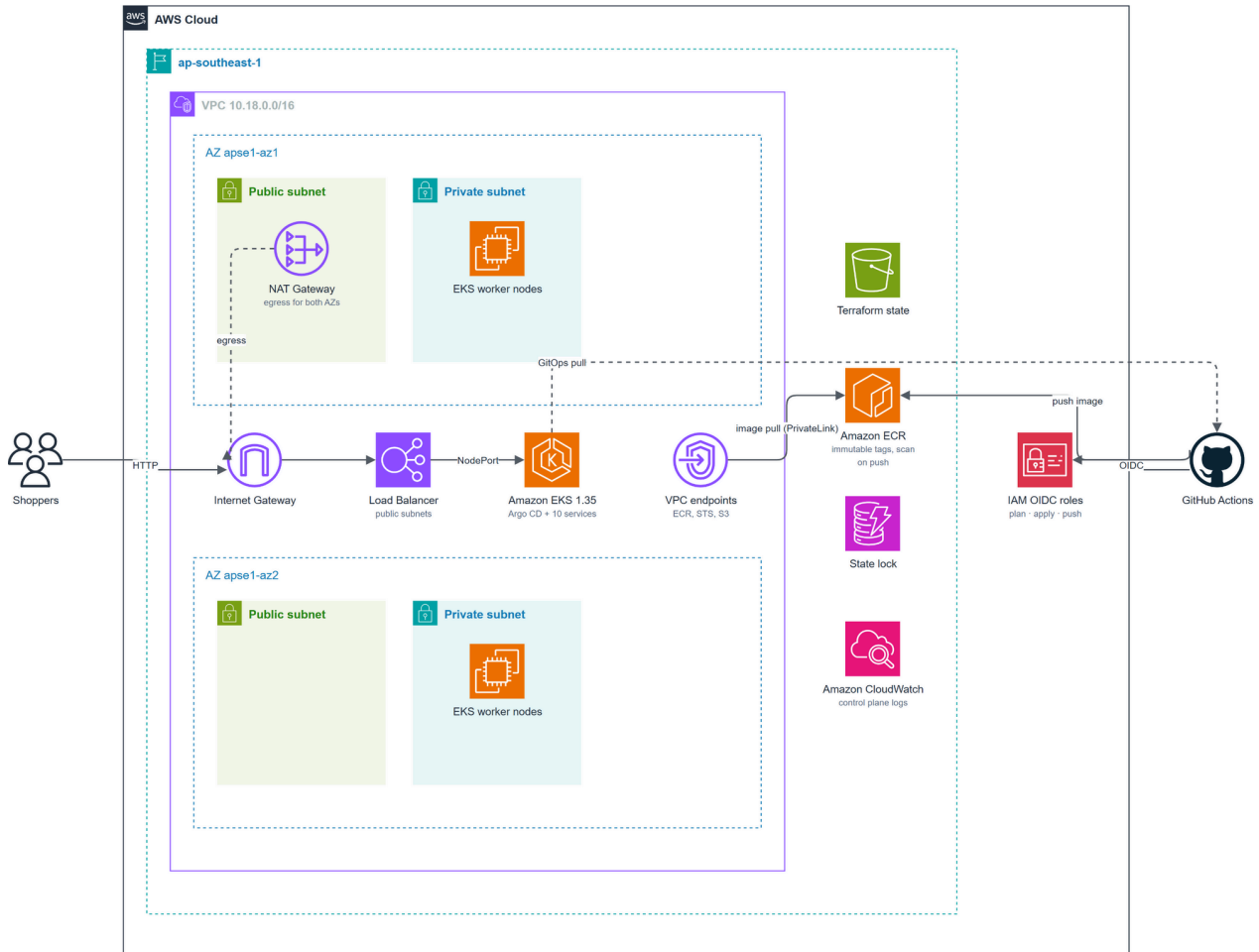


Figure 1: High-level architecture on AWS

Inbound traffic enters through the Internet Gateway to the load balancer in the public subnets, which forwards it to the worker nodes in the private subnets [AWS]. The nodes reach ECR, STS and S3 through VPC endpoints and the rest of the internet through one NAT Gateway. Argo CD inside the cluster pulls the desired state from GitHub.

3 Requirements and constraints

3.1 Non-functional requirements

Area	Requirement	How it is met
Security	No long-lived AWS credentials anywhere in the repository or in GitHub secrets.	GitHub OIDC roles, IRSA for the VPC CNI.
Security	Only reviewed code reaches AWS; pull requests cannot change infrastructure.	Read-only plan role for PRs; apply needs the production environment approval.
Security	Misconfigured infrastructure and vulnerable images are detected before deployment.	Checkov gate before plan/apply; Trivy before push.
Security	A compromised pod cannot reach services it does not call.	Pod hardening, PSA restricted, NetworkPolicy per service.
Availability	The entry point survives the loss of one pod or one node.	frontend runs 2-4 replicas with a PodDisruptionBudget and zone/node spreading.
Operability	The cluster state can be rebuilt from Git alone.	Argo CD with automated sync, prune and self-heal.
Portability	Moving to another AWS account needs no code change.	Account ID read from credentials (Terraform) and the AWS_ACCOUNT_ID variable (CI).
Cost	A dev environment that can be created and destroyed on demand.	One NAT Gateway, 2 t3.medium nodes, terraform destroy documented.

3.2 Constraints

- **No changes to src/.** The application is upstream code. Lint and vulnerability findings that need code changes stay visible as warnings instead of being hidden.
- **Single region, single environment.** Everything runs in ap-southeast-1 as dev.
- **Personal GitHub repository.** Repository rulesets cannot list the GitHub Actions app as a bypass actor (only organisation repositories can), which affects the GitOps write-back (see Section 10).
- **No AWS account at the time of writing.** CI skips every AWS step while the AWS_ACCOUNT_ID repository variable is unset.

4 AWS infrastructure design

4.1 Network

Table 2: Subnet plan, VPC 10.18.0.0/16 (cidrsubnet(vpc_cidr, 8, n))

Subnet	AZ ID	CIDR	Hosts
public (az1)	apse1-az1	10.18.0.0/24	NAT Gateway, load balancer
public (az2)	apse1-az2	10.18.1.0/24	load balancer
private (az1)	apse1-az1	10.18.2.0/24	EKS nodes, interface endpoints
private (az2)	apse1-az2	10.18.3.0/24	EKS nodes, interface endpoints

- Subnets are placed by **AZ ID** (apse1-az1, apse1-az2) rather than AZ name, because AZ names map to different physical zones in every account.
- The public route table sends 0.0.0.0/0 to the Internet Gateway. The private route table sends 0.0.0.0/0 to the NAT Gateway and S3 prefixes to the S3 gateway endpoint.
- **One NAT Gateway** in public subnet az1 serves both AZs. This is a cost decision for dev: if az1 fails, nodes in az2 lose internet egress (they keep ECR/STS/S3 access through the endpoints). See decision D3.
- VPC endpoints: interface endpoints ecr.api, ecr.dkr and sts with private DNS, and a gateway endpoint for S3 (ECR stores image layers in S3). Image pulls and IRSA token exchange therefore stay inside the VPC [AWS].
- Network ACLs allow all traffic; filtering is done with security groups and Kubernetes NetworkPolicy. The open NACLs are accepted Checkov findings (Section 7.2).

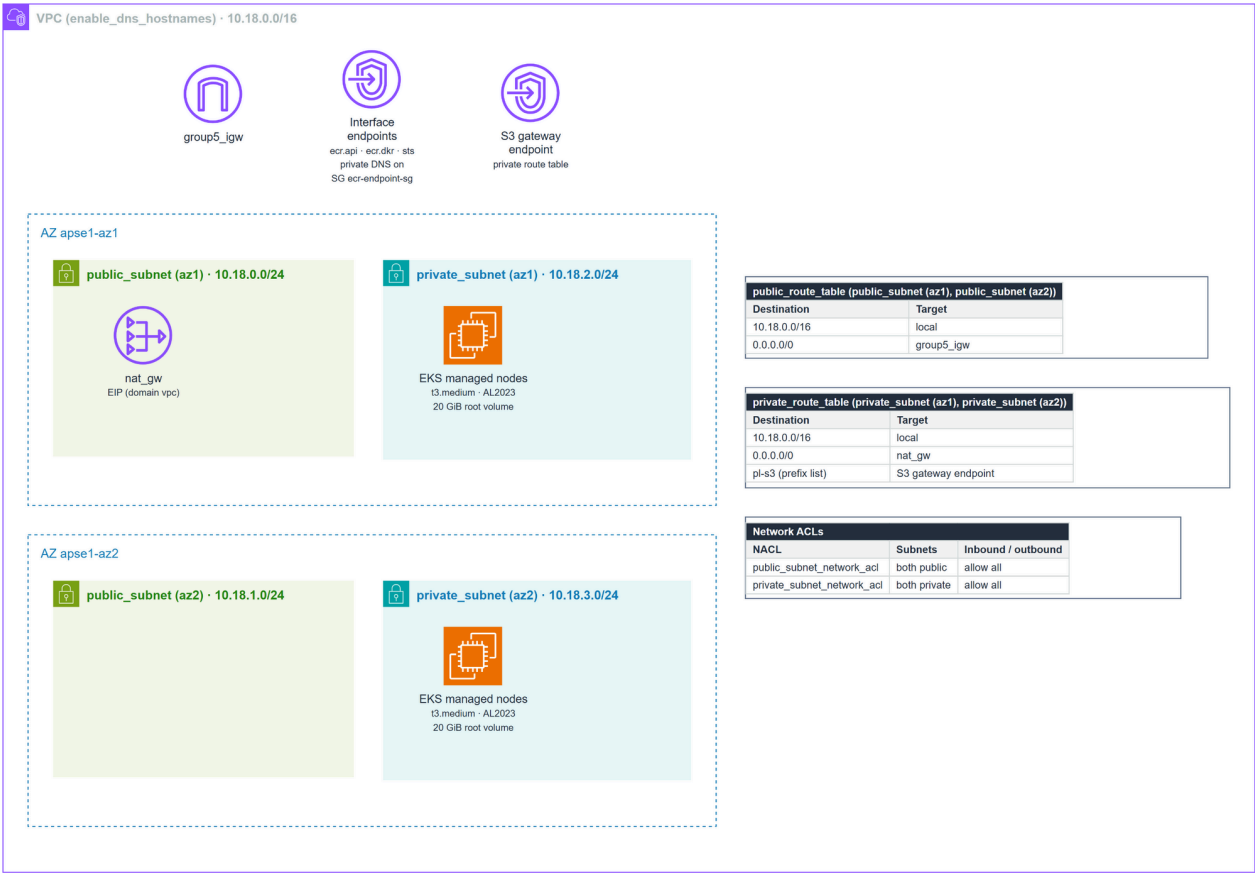
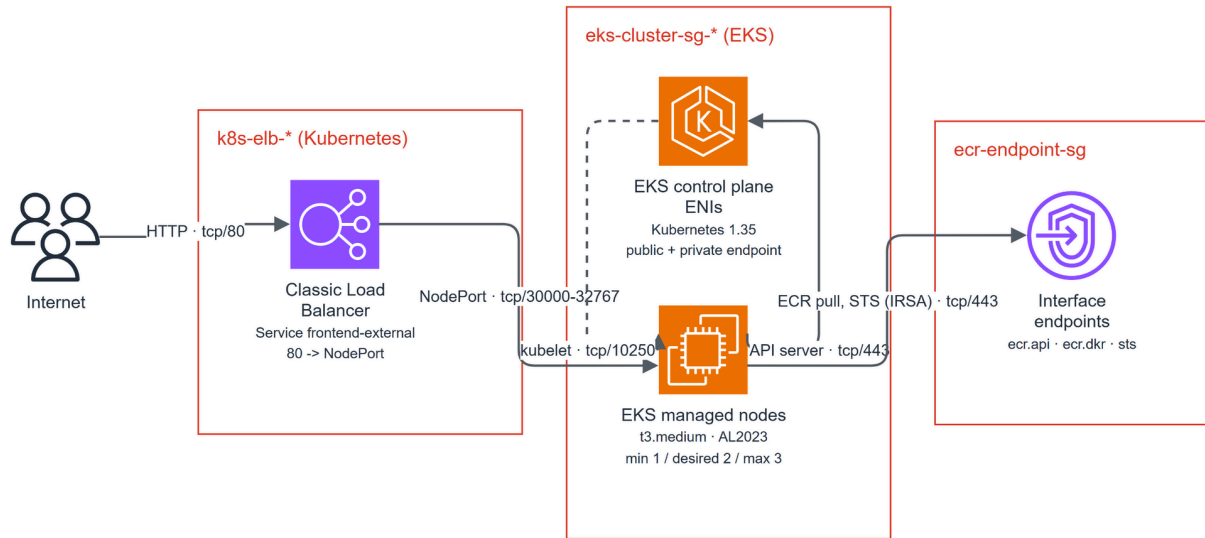


Figure 2: LLD: network detail

4.2 Security groups



Security group inbound rules			
Security group	Port	Source	Description
k8s-elb-* (Kubernetes)	tcp/80	0.0.0.0/0	frontend-external listener
eks-cluster-sg-* (EKS)	all	eks-cluster-sg-* (EKS)	EKS default: nodes and control plane
eks-cluster-sg-* (EKS)	tcp/30000-32767	k8s-elb-* (Kubernetes)	NodePort, added by Kubernetes
ecr-endpoint-sg	all	10.18.0.0/16	whole VPC (only tcp/443 is needed)

Figure 3: LLD: security group flow

- k8s-elb-* (created by Kubernetes for the LoadBalancer Service) accepts tcp/80 from the internet.
- The EKS cluster security group allows all traffic between nodes and control plane, plus the NodePort range tcp/30000-32767 from the load balancer (added by Kubernetes).
- ecr-endpoint-sg currently accepts all protocols from the VPC CIDR; only tcp/443 is needed. Narrowing it is on the roadmap.

4.3 Compute: Amazon EKS

Setting	Value
Cluster	khaipd18-eks-cluster, Kubernetes 1.35
API endpoint	Public and private. Public access is open to 0.0.0.0/0 (accepted for dev, see Section 7.2) [AWS]
Control plane logs	All 5 types: api, audit, authenticator, controllerManager, scheduler
Node group	Managed, t3.medium, AL2023_x86_64_STANDARD, on-demand, 20 GiB disk, min 1 / desired 2 / max 3, private subnets only
Add-ons	VPC CNI with enableNetworkPolicy and its own IRSA role, CoreDNS, kube-proxy, Metrics Server (community add-on)
Cluster access	No access_config block: the API default applies (authentication mode CONFIG_MAP, creator gets admin) [AWS]

Kubernetes 1.35 reaches end of standard support on 27 March 2027 and end of extended support on 27 March 2028 [\[AWS\]](#). The upgrade procedure is in the runbook.

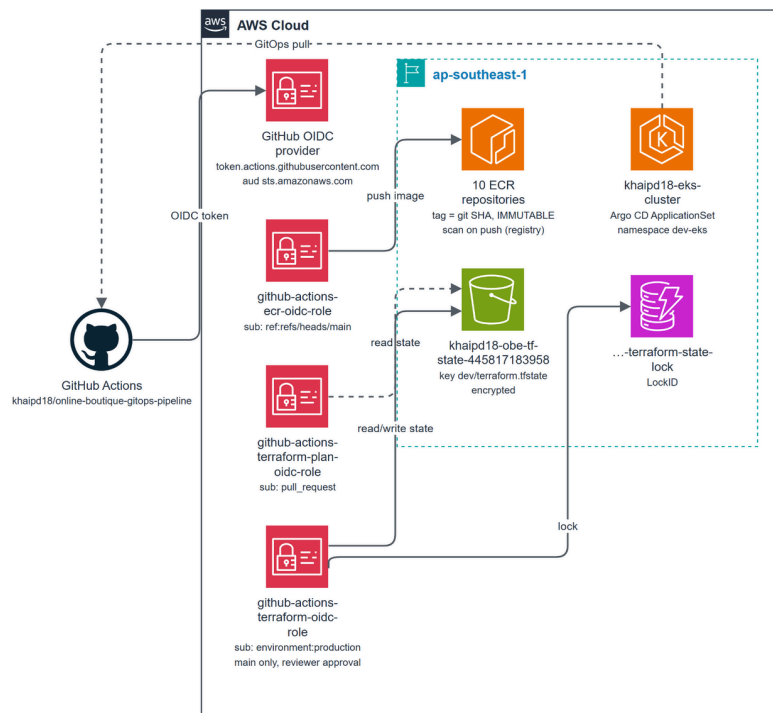
NOTE

Because no access_config is set, only the IAM principal that created the cluster (the one that ran the first terraform apply) can use kubectl at first. Setting authentication_mode = "API_AND_CONFIG_MAP" and managing access entries in Terraform is the recommended next step [\[AWS\]](#).

4.4 Container registry: Amazon ECR

- 10 repositories, one per service, with IMMUTABLE tags: an image tag (the git SHA) can never be overwritten [AWS].
- Lifecycle policy: untagged images expire after 14 days; images not pulled for 90 days move to the archive storage class.
- Scan on push (basic scanning) is configured at **registry** level with `aws_ecr_registry_scanning_configuration`, as AWS recommends instead of the deprecated repository-level setting [AWS].
- `force_delete = true` lets terraform destroy remove repositories that still hold images. This suits a disposable dev environment and should be turned off for production.

4.5 Identity and access



GitHub Actions roles			
Role	Trusted sub (repo:<repo>:...)	Permissions	Used by
github-actions-ecr-oidc-role	ref:refs/heads/main	ECR push/pull on the 10 repositories	5 service CI workflows
github-actions-terraform-plan-oidc-role	pull_request	ReadOnlyAccess + state read (plan-lock=false)	terraform.yaml on pull requests
github-actions-terraform-oidc-role	environment:production	AdministratorAccess + state read/write/lock	terraform.yaml apply job, push to main + approval

Figure 4: LLD: CI/CD and IAM OIDC

GitHub Actions authenticates with short-lived tokens from the GitHub OIDC provider `token.actions.githubusercontent.com`. Each role trusts an exact sub claim [AWS]:

Table 3: GitHub Actions roles (terraform/main.tf)

Role	Trusted sub (after repo:<repo>:)	Permissions
github-actions-ecr-oidc-role	ref:refs/heads/main	GitHubActions-ECR-Push-Policy (push/pull on the 10 repositories)
github-actions-terraform-plan-oidc-role	pull_request	ReadOnlyAccess + GitHubActions-Terraform-State-Read-Policy
github-actions-terraform-oidc-role	environment:production	AdministratorAccess + GitHubActions-Terraform-State-Policy

- Both GitHub sub formats are trusted: the classic `owner/repo` and the immutable `owner@id/repo@id` format that GitHub uses for repositories renamed after 2026-07-15.
- The apply role trusts the `production` **environment** rather than the `main` branch. That environment accepts deployments from `main` only and waits for a reviewer, as AWS recommends when a trust policy relies on GitHub environments [\[AWS\]](#).
- GitHub does not issue OIDC tokens to pull requests from forks, so the plan role can only be used by collaborators with write access.
- Inside the cluster, the VPC CNI uses IRSA (role bound to `kube-system/aws-node`) instead of the node role [\[AWS\]](#). No application pod needs AWS access.

4.6 Terraform state

- State: S3 bucket `khaipd18-obe-tf-state-445817183958`, key `dev/terraform.tfstate`, encrypted, versioning enabled.
- Locking: DynamoDB table `khaipd18-devops-project-terraform-state-lock` (LockID). The plan role cannot write to it, so PR plans run with `-lock=false`.
- The bucket and table are created by hand once (bootstrap) and are not managed by Terraform.

4.7 Infrastructure as Code structure

Module	Creates
<code>vpc</code>	VPC, 2 public + 2 private subnets, Internet Gateway, NAT Gateway, route tables, NACLs (child modules <code>subnet</code> , <code>igw</code> , <code>nat_gw</code> , <code>route-table</code>)
<code>eks</code>	Cluster, control plane log group (365-day retention), managed node group, 4 add-ons, OIDC provider and IRSA role for the VPC CNI
<code>ecr</code>	10 repositories, lifecycle policies, registry scanning configuration
<code>vpc-endpoints</code>	Interface endpoints <code>ecr.api</code> , <code>ecr.dkr</code> , <code>sts</code> ; S3 gateway endpoint; <code>ecr-endpoint-sg</code>
<code>github-oidc-role</code>	IAM role with a trust policy built from a list of repositories and allowed sub values

The root module wires the modules together and defines the GitHub OIDC provider, the three roles and their policies. The account ID is never hardcoded: Terraform reads it from the active credentials, and the workflows read it from the `AWS_ACCOUNT_ID` repository variable.

5 Kubernetes platform design

5.1 Namespaces and Pod Security

- All workloads run in dev-eks. The namespace is managed by its own Argo CD Application (namespaces) with Delete=false, so a sync can never delete it.
- Pod Security Admission enforces the restricted profile. A pod without a compliant security context is rejected at admission.
- Monitoring runs in monitoring; Argo CD in argocd.

5.2 Universal Helm chart

All 12 releases use helm-charts/ (standard-microservice). A service's values file only declares what differs. The chart is secure by default:

Value	Default and meaning
podSecurityContext	runAsNonRoot, UID/GID 10001, seccompProfile: RuntimeDefault
securityContext	No privilege escalation, not privileged, read-only root filesystem, drop ALL capabilities
automountServiceAccountToken	false: no service calls the Kubernetes API
networkPolicy.allowFrom	Release names allowed to connect; empty means deny all ingress
networkPolicy.allowFromAnywhere	Open the service port to any source (used by frontend)
autoscaling.*	CPU HPA, off by default; min 2, max 4, target 70 %
podDisruptionBudget.*	minAvailable: 1, rendered only when the service runs 2 or more replicas
topologySpread.enabled	Soft spread across zones, then nodes (ScheduleAnyway)
service.selectorOverride	Point a Service at another release's pods (used by frontend-external)

frontend-external is a release with deployment.enabled: false that only creates the LoadBalancer Service (80 → 8080) in front of the frontend-dev pods. Internet exposure is therefore separate from the workload.

5.3 Service-to-service traffic

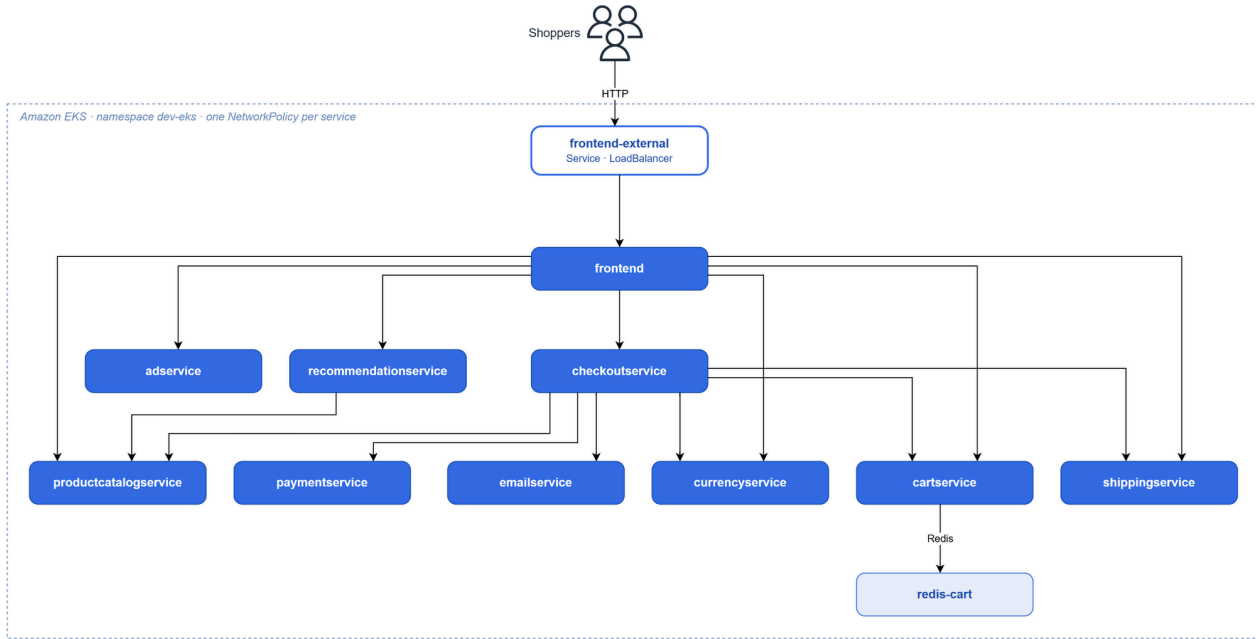


Figure 5: In-cluster calls allowed by NetworkPolicy

Each release gets an **ingress** NetworkPolicy that admits only the callers listed in `allowFrom`, on the service port only. The VPC CNI enforces the policies [AWS]. Egress is not restricted.

5.4 Resilience

- frontend (the only entry point) has a CPU HPA with 2 to 4 replicas. Metrics come from the Metrics Server add-on, which EKS does not install by default [AWS].
- A PodDisruptionBudget keeps at least one frontend pod during node drains and upgrades.
- Replicas spread across zones and then nodes when the scheduler can place them there.
- The other services run 1 replica. `redis-cart` must stay at 1: it uses an `emptyDir` volume, so the cart is lost when its pod restarts.

5.5 Observability

`kube-prometheus-stack 84.4.0` (Prometheus, Grafana, Alertmanager, node exporter, kube-state-metrics) is deployed by Argo CD into monitoring with `ServerSideApply=true` because its CRDs exceed the annotation size limit of client-side apply. EKS control plane logs go to CloudWatch Logs, in a log group that Terraform creates with a 365-day retention so that `terraform destroy` also removes it. There are no custom alert rules, dashboards per service, central log store or tracing yet.

6 Delivery pipeline design

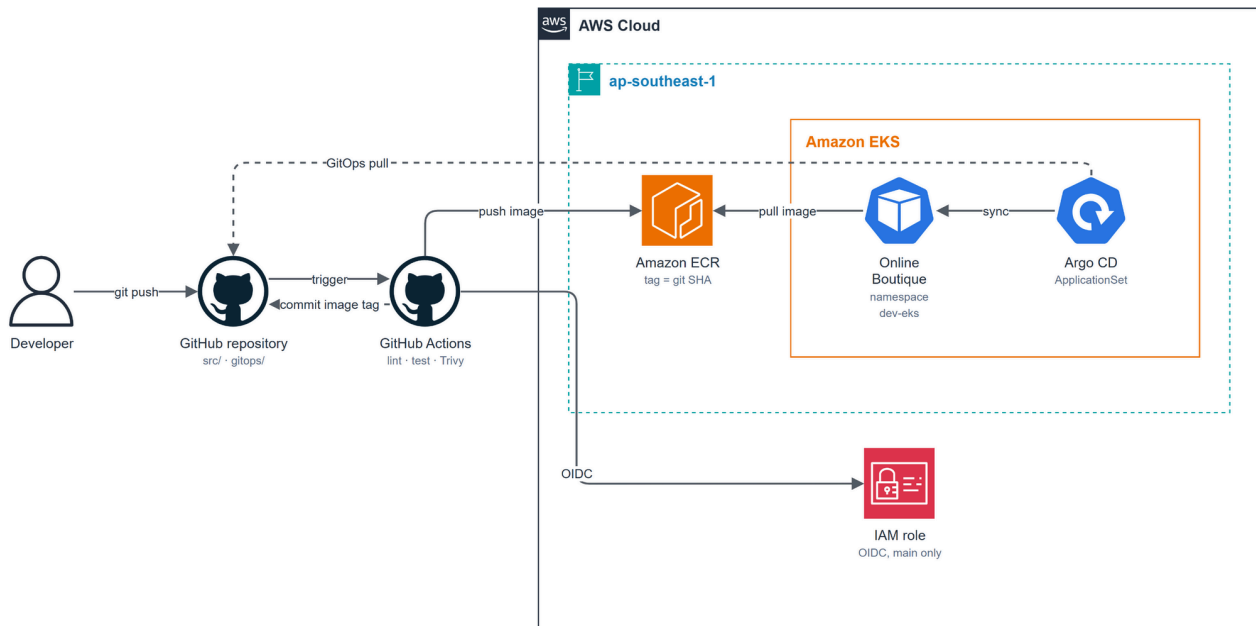


Figure 6: From commit to cluster

6.1 Continuous integration

Each language has its own workflow, triggered by pushes to `main` under that language's `src/` paths or by a manual run. `doorny/paths-filter` builds a matrix of the services that changed, and each service runs:

1. Lint, unit tests and dependency scan (Table 4).
2. Log in to AWS with the ECR role, then check whether the image for this commit already exists (tags are immutable, so it is never rebuilt).
3. Build the image and scan it with Trivy (composite action `.github/actions/trivy-scan`): HIGH/CRITICAL findings go to GitHub code scanning (SARIF) and a CycloneDX SBOM is stored as an artifact.
4. Push the image to ECR, tagged with the git SHA.
5. Write `image.repository` and `image.tag` into `gitops/dev-eks/values-<service>.yaml` and push the commit with `[skip ci]` (retry loop for concurrent services).

While `AWS_ACCOUNT_ID` is unset, steps 2, 4 and 5 are skipped; the image is built as `local/<service>:<sha>` and still scanned.

Table 4: Quality gates

Stack	Lint / format	Test	Dependency scan
Go (4 services)	golangci-lint v1.64.8 (blocking)	go test (blocking)	govulncheck v1.8.0 (warning)
.NET (cartservice)	dotnet format (warning)	dotnet test (blocking)	dotnet list package --vulnerable (blocking)
Java (adservice)	google-java-format (warning)	gradle test (blocking)	–
Node.js (2 services)	ESLint 8 (blocking)	–	npm audit (warning)
Python (2 services)	flake8 (blocking)	pytest (blocking when tests exist)	bandit (blocking)

Steps marked **warning** can only be fixed by changing the upstream code. They use `continue-on-error` so they show as a warning on every run instead of being hidden. Trivy runs in report mode (`blocking: 'false'`) for the same reason.

6.2 Continuous delivery

One ApplicationSet (`microservices-dev-eks`) generates 12 Applications named `<service>-dev` from a list generator. Each renders `helm-charts/` with `gitops/dev-eks/values-<service>.yaml`, targets namespace `dev-eks`, and syncs automatically with `prune` and `selfHeal`. A manual change in the cluster is reverted to what Git says. Adding a service means adding one values file and one list element.

6.3 Infrastructure pipeline

`terraform.yaml` runs on pull requests and pushes that change `terraform/` or the workflow itself:

Job	Behaviour
Checkov Scan	Always runs. Blocking: any finding that is not in <code>terraform/.checkov.baseline</code> fails the run.
Terraform Plan	Pull requests from this repository only. Read-only role, <code>fmt -check, validate, plan -lock=false</code> .
Terraform Apply	Pushes to <code>main</code> only. Uses environment <code>production</code> , so it waits for a reviewer before it gets an OIDC token; then <code>apply -auto-approve</code> .

Plan and Apply are skipped while `AWS_ACCOUNT_ID` is unset. The very first apply in a new account runs from a workstation, because the GitHub roles do not exist yet.

6.4 Repository controls

- Ruleset `protect-main`: no deletion, no force push, pull request required; repository admins may bypass. No status check is required, because every workflow is path-filtered and a required check that never starts would block the pull request.
- Environment `production`: deployment branch `main` only, required reviewer `khaipd18`.
- `GITHUB_TOKEN` defaults to contents: `read`; jobs request more only where needed (`id-token: write`, `security-events: write`, contents: `write` for the write-back).

7 Security design

7.1 Controls by risk

Risk	Control	Where
Leaked access keys	OIDC for GitHub Actions, IRSA in the cluster; no keys stored anywhere	terraform/main.tf, modules/eks/iam.tf
A branch or PR gains AWS admin	Apply role trusts only environment production (main + reviewer); ECR role only main; PRs get read-only plan	modules/github-oidc-role
Misconfigured Terraform	Checkov blocks before plan/apply; reviewed findings in a baseline	.checkov.yaml, terraform/.checkov.baseline
Vulnerable images	Trivy before push, results in code scanning, SBOM per image	.github/actions/trivy-scan
Overwritten images	Immutable tags, tag = git SHA	modules/ecr
Compromised container	Non-root, read-only root FS, drop ALL, seccomp, no service account token	helm-charts/values.yaml
Non-compliant pods	PSA restricted in enforce mode	gitops/namespaces/dev-eks.yaml
Lateral movement	Ingress NetworkPolicy per service following the gRPC call graph	helm-charts/templates/networkpolicy.yaml
Over-privileged CI token	contents: read by default	.github/workflows/*
Unreviewed changes on main	Ruleset protect-main	GitHub settings

7.2 Accepted exceptions

Terraform findings reviewed and kept in terraform/.checkov.baseline (13 in total):

Check	Finding	Reason / plan
CKV_AWS_39	EKS public endpoint enabled	Needed for kubectl and CI without a VPN; restrict CIDRs or go private (roadmap)
CKV_AWS_58	No customer managed KMS key for EKS secrets	EKS 1.28+ already envelope-encrypts all Kubernetes API data with an AWS owned key [AWS] add a customer managed key if key control or audit is required
CKV_AWS_229-232	NACLs allow ports 20, 21, 22, 3389 (x2 NACLs)	Filtering is done by security groups and NetworkPolicy
CKV2_AWS_11	VPC flow logs disabled	Cost; enable for production
CKV2_AWS_12	Default security group not restricted	Unused by any resource; restrict for production
CKV_AWS_158	Control plane log group not encrypted with a customer managed KMS key	CloudWatch Logs encrypts every log group at rest by default [AWS] add a KMS key if key control is required

Checks skipped globally in .checkov.yaml: CKV_AWS_163 (scan on push is set at registry level, which the check does not see), CKV_AWS_136 (ECR uses AES-256 encryption by default), CKV2_AWS_1 (false positive with aws_network_acl_association).

Kubernetes manifests rendered from the chart have 12 remaining Checkov findings (from 128 before hardening): images not pinned by digest (11) and the Redis pull policy (1). GitHub Actions has 5 findings on the free-text custom_tag input of workflow_dispatch.

7.3 Supply chain

- Images are scanned before push; known HIGH/CRITICAL vulnerabilities with a fix are counted per image on every run and shown in the GitHub **Security** tab.
- Tool versions are pinned (Checkov, Trivy, golangci-lint, govulncheck, Terraform, AWS provider). GitHub Actions are pinned by major version, not by commit SHA.
- Images are not signed and not pinned by digest yet (roadmap).

8 Design decisions

ID	Decision	Rationale	Revisit when
D1	GitHub OIDC roles instead of access keys	No secret to leak or rotate; trust can be scoped to a branch, a PR or an environment.	–
D2	Apply gated by a GitHub environment	A merge alone cannot change infrastructure; the gate shows up in the GitHub UI with an audit trail.	More reviewers join (enable “prevent self-review”).
D3	One NAT Gateway for both AZs	NAT Gateways are billed per hour and per GB [AWS] acceptable single point of failure for dev.	Production: use a regional NAT gateway, which spans AZs on its own and needs no public subnet [AWS] .
D4	One universal Helm chart	Security defaults defined once; a new service is one values file.	A service needs resources the chart cannot express.
D5	ApplicationSet with a list generator	Explicit list of what is deployed, reviewed in Git.	Several environments (switch to a matrix or git generator).
D6	Image tag = git SHA, immutable ECR tags	Every running image maps to one commit; rollback = previous SHA.	–
D7	Classic Load Balancer from a Service of type LoadBalancer	No extra controller to install for dev.	TLS, path routing or WAF are needed (AWS Load Balancer Controller).
D8	IRSA for the VPC CNI	Works on every EKS version and was in place first.	EKS Pod Identity is now the simpler option: no OIDC provider per cluster and a single trust principal [AWS] .
D9	DynamoDB state locking	Supported by every Terraform version in use when the project started.	Move to S3 native locking (roadmap).
D10	Upstream findings as warnings, not fixes	The application code is not maintained here.	Dependencies are upgraded (then set gates to blocking).

9 Verification

Check	Result
Run on Amazon EKS (May 2026)	Infrastructure built by Terraform, images pushed over OIDC, services synced by Argo CD. Evidence: bot commits 14bc93f, eb86a9a, 46e66d5.
End-to-end on kind (Kubernetes 1.37)	11/11 pods ready with the chart's security context; browse, cart, currency change and checkout work; PSA rejects a non-compliant pod; NetworkPolicy blocks 3/3 unauthorised connections.
HPA and PDB on a 3-node kind cluster	frontend scaled 2 → 3 at 82 % CPU; draining the last node that ran frontend was blocked by the PDB.
Apply on a new account (2026-10-10)	73 resources created in 17 minutes; 2 nodes Ready in two AZs, 4 add-ons ACTIVE, Metrics Server serving metrics; terraform destroy removed everything in 7 minutes.
Terraform	fmt and validate pass; plan against an empty state: 74 resources to add, no errors.
Checkov 3.3.22	Terraform 0 new findings (12 baselined); Kubernetes 999 passed / 12 failed; GitHub Actions 5 failed.
CI without AWS (2026-10-08)	All 5 language workflows green; AWS steps skipped; Trivy results uploaded for all 10 images.

10 Risks, limitations and roadmap

10.1 Known risks

Risk	Impact	Mitigation today
CI bot blocked by the main ruleset	Once AWS is configured, the GitOps write-back push is rejected and new images are not deployed.	None yet: needs a deploy key on the bypass list or PR-based write-back.
Single NAT Gateway	Loss of az1 removes internet egress for both AZs.	ECR/STS/S3 traffic uses endpoints.
Single replica for 10 services	A pod restart causes a short outage of that feature.	Kubernetes restarts pods; frontend has 2+ replicas.
Cart data in emptyDir	Carts are lost when redis-cart restarts.	Accepted for a demo.
Known image vulnerabilities	Several images have CRITICAL findings with a fix available.	Visible in the Security tab; gates can be switched to blocking.
Public EKS endpoint open to the internet	API server reachable from anywhere (still needs IAM auth).	IAM authentication; audit logs enabled.

10.2 Roadmap

In order of priority:

1. Restore the GitOps write-back under the main ruleset (deploy key on the bypass list, or the bot opens pull requests).
2. Upgrade service dependencies (Dependabot), then make Trivy, govulncheck and npm audit blocking.
3. Restrict the EKS public endpoint or move to a private endpoint.
4. Narrow `ecr-endpoint-sg` to `tcp/443` from the VPC.
5. Replace the single NAT Gateway with a regional NAT gateway.
6. Manage cluster access with EKS access entries in Terraform.
7. External Secrets Operator with AWS Secrets Manager.
8. Pin images by digest, sign them with cosign and verify at admission.
9. AWS Load Balancer Controller with TLS instead of the Classic Load Balancer.
10. Staging and production environments with promotion, canary releases with Argo Rollouts.
11. SLO-based alert rules and per-service Grafana dashboards.
12. S3 native state locking instead of DynamoDB; consider EKS Pod Identity instead of IRSA.

Appendix A. Versions

Component	Version	Defined in
Terraform	1.14.8	terraform.yaml, README
AWS provider	6.39.0	terraform/versions.tf
Kubernetes (EKS)	1.35	terraform/variables.tf
VPC CNI add-on	v1.21.1-eksbuild.7	terraform/variables.tf
CoreDNS add-on	v1.13.2-eksbuild.4	terraform/variables.tf
kube-proxy add-on	v1.35.3-eksbuild.2	terraform/variables.tf
Metrics Server add-on	v0.9.0-eksbuild.11	terraform/variables.tf
kube-prometheus-stack	84.4.0	gitops/argocd/monitoring.yaml
Argo CD	stable manifest	README, step 3
Checkov	3.3.22	.github/workflows/*
Trivy	0.75.0	.github/actions/trivy-scan
Redis (cart store)	8.10.2	gitops/dev-eks/values-redis-cart.yaml

Appendix B. Glossary

Term	Meaning
ApplicationSet	Argo CD resource that generates several Applications from a template and a generator.
GitOps write-back	CI commits the new image tag to Git so that Argo CD deploys it.
HPA / PDB	HorizontalPodAutoscaler / PodDisruptionBudget.
IRSA	IAM Roles for Service Accounts: pods assume IAM roles through the cluster's OIDC provider.
OIDC	OpenID Connect. GitHub issues a signed token per job; AWS STS exchanges it for temporary credentials.
PSA	Pod Security Admission, the built-in admission controller for the Pod Security Standards.
SBOM	Software Bill of Materials; here in CycloneDX format.
sub claim	Subject of the GitHub OIDC token, e.g. <code>repo:owner/repo:ref:refs/heads/main</code> .